Introduction to graph theory

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Formal graph models

A finite simple undirected graph is $G = (V, E)$, $E \subseteq \binom{V}{2}$, where the latter consists of unordered pairs of distinct vertices. This excludes loops and parallel edges. A multigraph distinguishes parallel edges by IDs; an undirected loop contributes two to degree. Euler's theorem still applies.

Order is |V| and size |E|. Adjacency concerns endpoints, not their drawn positions. Degree counts incident endpoints with multiplicity. A simple directed graph uses ordered pairs (u, v),u≠v; outdegree d^+ counts departing arcs and indegree d^- arriving arcs. Specify simple/multi, directed/undirected, labeled/unlabeled, and weighted/unweighted models first.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Walks, trails, paths, and cycles

A walk alternates incident vertices and edges; length counts edge uses. A trail repeats no edge ID. A simple path repeats no vertex. In a simple undirected graph a cycle has length at least three, equal first/last vertex, and no other repetition. A closed trail may repeat other vertices and need not be a cycle.

A length-zero walk at one vertex is also a path. Multigraph cycle-length conventions require separate specification. An Euler circuit is a closed trail and is not limited by the simple-cycle minimum length. Undirected path connectivity is reflexive, symmetric, and transitive; its classes are components. One component means connected. A simple path on n vertices has length at most n−1. Euler trails use every edge once; Hamilton paths every vertex once. Their existence conditions differ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Adjacency matrices count walks

In a fixed vertex order, $A_{ij}=1$ iff v_iv_j is an edge. For a simple undirected graph $A=A^{\mathsf{T}}$, diagonal zero, and row sums are degrees. Directed matrices need not be symmetric.

The entry $(A^r)_{ij}$ counts length-r walks from v_i to v_j. At r=0, $A^0=I$ counts empty walks. Inductively

$$(A^{r+1})_{ij} = \sum_k (A^r)_{ik} A_{kj}$$

classifies walks by their penultimate vertex. Repeated vertices and edges are permitted; powers do not count simple paths. For path 1–2–3,

$$A^2 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}.$$

The central two counts 2–1–2 and 2–3–2, neither a simple path.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Degree sums and the handshake lemma

$\sum_{v \in V} \deg(v) = 2|E|$, because each edge contributes at both ends. Thus the number of odd-degree vertices is even; this necessary condition does not characterize graphical degree sequences. Sequence (3,1,1) has odd total five and is impossible; (3,3,1,1) has even sum but needs further testing.

Count parallel edges separately and loops twice, preserving the identity for finite multigraphs. In a digraph each arc contributes one outdegree and one indegree, so both sums equal the arc count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subgraphs, complements, and isomorphisms

A subgraph H=(W, F) has W⊆V, F⊆E with every retained edge's endpoints in W. The induced subgraph retains all edges between W's vertices. A simple graph's complement uses the same vertices and exactly the absent distinct-vertex edges.

An isomorphism is a bijection $\varphi : V(G) \to V(H)$ preserving adjacency in both directions. Compare order, size, degree multiset, triangles, components first; matching invariants alone is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal degrees do not prove isomorphism

C_6 and two disjoint triangles each have six vertices, six edges, and all degrees two. One is connected and the other has two components, so they are nonisomorphic. Even matching connectivity would not eliminate all possible distinctions.

An isomorphism's permutation matrix satisfies $A_H = P^T A_G P$; conversely this entrywise equality proves adjacency preservation. Equal spectra alone generally do not suffice. For complements, $A_{\overline{G}} = J - I - A_G$ and $d_{\overline{G}}(v) = n - 1 - d_G(v)$. The edge sets partition K_n. The complement of 1–2–3 has only edge 13 and isolated 2; flipping every matrix entry would wrongly introduce loops.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler trails and circuits

When all nonisolated vertices are connected, an Euler circuit exists iff every degree is even. An open Euler trail exists iff exactly two degrees are odd, at its endpoints. Necessity follows from paired arrivals/departures at internal vertices.

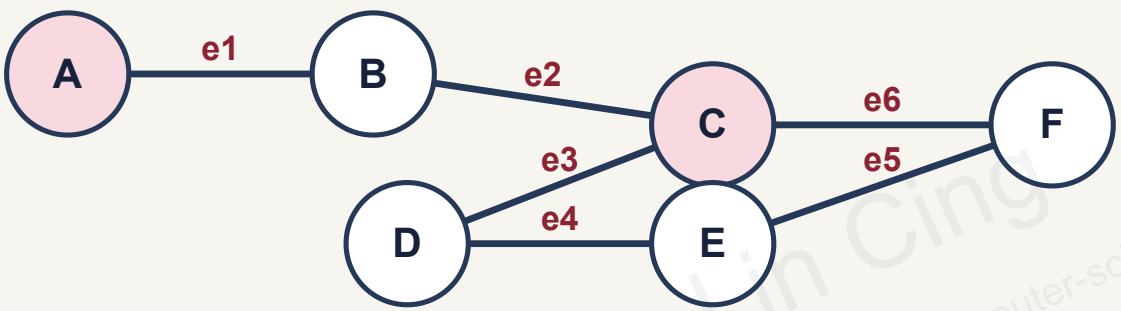
For sufficiency in the even case, follow unused edges from a nonisolated start. No other vertex can be the first dead end, so the trail closes. If edges remain, connectivity supplies a current-trail vertex incident to unused edges; form another closed trail there and splice it in. Each step adds an edge, so finitely many steps exhaust all edges. With two odd vertices, add a temporary edge between them, find a circuit in the even multigraph, then remove that edge to obtain the required trail.

For $e_1,\ldots,e_6$=AB, BC, CD, DE, EF, FC, the degrees are A:1,B:2,C:3,D:2,E:2,F:2. Only A, C are odd.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler trails and circuits (continued)

Count incident edges at every vertex



Vertex degrees

A=1 C=3

B=D=E=F=2

Only A,C have odd degree

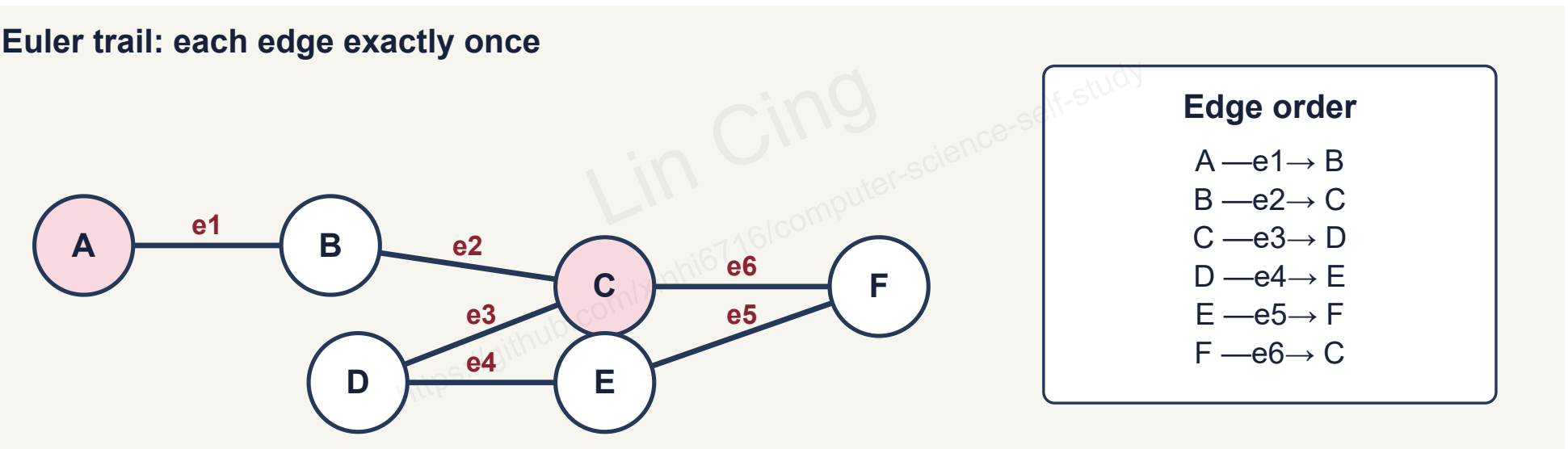
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every edge in an Euler trail

The figure pairs the same graph with its six-row traversal. Each row uses one edge ID, and every ID appears once:

$$A \xrightarrow{e_1} B \xrightarrow{e_2} C \xrightarrow{e_3} D \xrightarrow{e_4} E \xrightarrow{e_5} F \xrightarrow{e_6} C.$$

Every internal visit pairs an arrival/departure; A, C each retain one unmatched incident use.



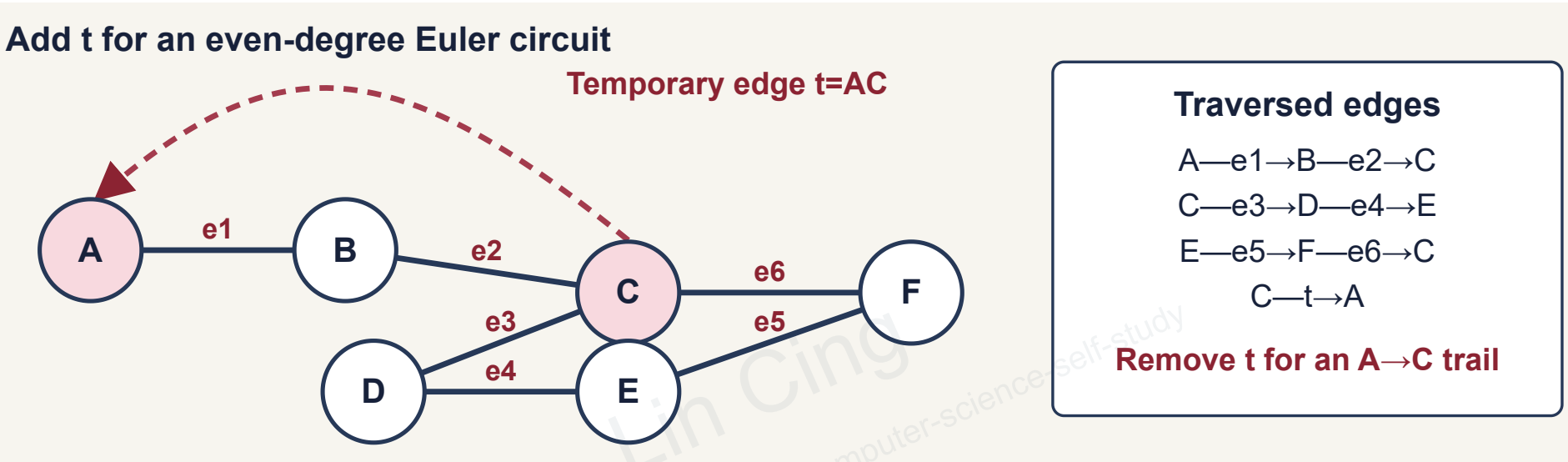
Temporary edge and cutting the circuit

Adding t=AC makes both odd degrees even. The circuit is A–B–C–D–E–F–C–A, with edges e_1,e_2,e_3,e_4,e_5,e_6,t. Removing its final temporary C→A edge gives the A-to-C trail.

Generally cut the circuit immediately after the temporary-edge traversal, then remove it. The resulting trail starts at that traversal's destination. Reverse an undirected trail if the endpoint order should be exchanged. This reduces odd endpoints to a circuit; the next section demonstrates genuine circuit splicing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Temporary edge and cutting the circuit (continued)



Several disconnected edge-containing components cannot share one trail traversing all edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hierholzer's splicing invariant

In a finite even-degree undirected multigraph with connected nonisolated part, follow unused edges from s. Stopping at $v \neq s$ would mean an odd number of used incident edges there. Even total degree then leaves an unused edge, contradiction. Thus the first stop is s.

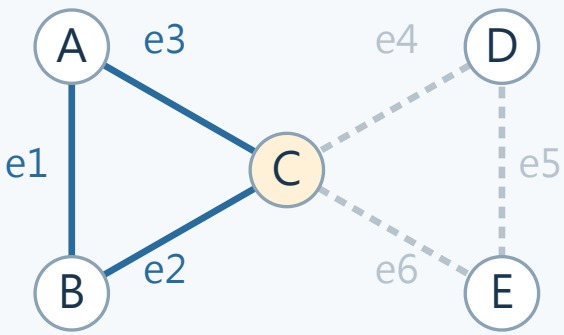
Removing a closed trail preserves even residual degrees. If unused edges remain, original connectivity gives a trail vertex x incident to them. Build another closed trail from x and insert it. The result remains closed, edge-distinct, and contains every previously used edge. Each insertion adds an edge, so all are eventually used.

For triangles sharing C with edges AB, BC, CA, CD, DE, EC, first take A–B–C–A, then C–D–E–C. Splicing at C yields A–B–C–D–E–C–A. C repeats but all six edges occur once.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hierholzer's splicing invariant (continued)

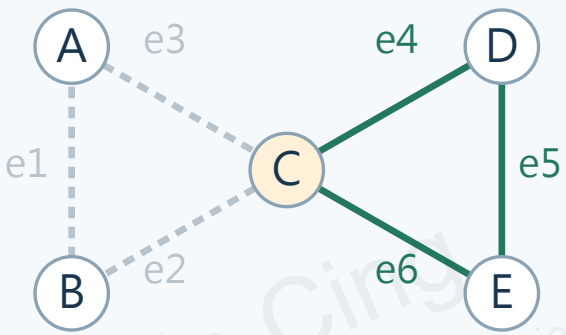
1 First closed trail



$A \to B \to C \to A$

e1, e2, e3

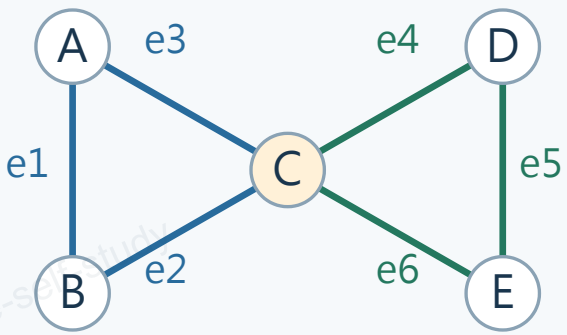
2 Unused edges at C



$C \to D \to E \to C$

e4, e5, e6

3 Insert at C



$A \to B \to C \to D \to E \to C \to A$

e1, e2, e4, e5, e6, e3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear-time Euler traversal

Assign one ID to each undirected edge and store it at both endpoint adjacency lists. Loops also use one used-ID flag. A per-vertex cursor skips already used IDs without rescanning the list.

```
S = [start]; answer = []
while S is not empty:
    v = S.top()
    advance cursor[v] past used edge IDs
    if v has no unused edge:
        answer.append(S.pop())
    else:
        take unused e=(v, w); used[e]=true
        S.push(w)
return reverse(answer)
```

The stack retains the not-yet-output trail; backtracking completes inner closed segments, and reversal realizes splicing. Check connectivity and parity first; an open trail starts at an odd vertex. Finally verify m used IDs and output length m+1. Each adjacency entry is scanned once, giving O(n+m) time and additional space.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A dead end at A does not end the search

Scan edge IDs in increasing order on the preceding graph. Initially S=[A], answer=[].

Action	S (top at right)	answer (append order)
e1	`[A,B]`	`[]`
e2	`[A,B,C]`	`[]`
e3	`[A,B,C,A]`	`[]`
pop A	`[A,B,C]`	`[A]`
e4	`[A,B,C,D]`	`[A]`
e5	`[A,B,C,D,E]`	`[A]`
e6	`[A,B,C,D,E,C]`	`[A]`

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pop A after e3; C still has e4, so continue. The first popped A becomes the **last** output vertex.

Backtracking writes the circuit in reverse

All six IDs are used. The remaining pops traverse no additional edge.

Action	S (top at right)	answer (append order)
pop C	`[A,B,C,D,E]`	`[A,C]`
pop E	`[A,B,C,D]`	`[A,C,E]`
pop D	`[A,B,C]`	`[A,C,E,D]`
pop C	`[A,B]`	`[A,C,E,D,C]`
pop B	`[A]`	`[A,C,E,D,C,B]`
pop A	`[]`	`[A,C,E,D,C,B,A]`

Reverse answer: **A–B–C–D–E–C–A**, using **e1,e2,e4,e5,e6,e3**. The second circuit is inserted before e3. Repeated C is legal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Directed Euler circuits

For m>0, a directed Euler circuit exists iff the underlying undirected nonisolated part is connected and $d^+(v)=d^-(v)$ everywhere. Necessity follows from paired arrivals/departures and one trail's connectivity.

For sufficiency, contract strongly connected components into a DAG. A source component has no entering arcs. Summing $d^+-d^-=0$ inside it cancels internal arcs, forcing no leaving arcs either. More than one component would contradict underlying connectivity; the nonisolated part is strongly connected.

Following unused outgoing arcs cannot stop away from the start: used arrivals exceed departures by one there, and total balance leaves an outgoing arc. Remove the resulting closed trail, retaining balance, and splice remaining trails as before. The same stack algorithm uses outgoing adjacency lists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Directed open trails and zero-edge cases

An Euler trail from s to distinct t exists iff the nonisolated underlying graph is connected, $d^+(s) - d^-(s) = 1, d^-(t) - d^+(t) = 1$, and every other vertex is balanced. Necessity counts arrivals/departures. Add temporary $t \to s$ for sufficiency, find a balanced circuit, cut after that arc and remove it.

For $A \to B$, $B \to C$, $C \to A$, $A \to C$, (out, in) pairs are $A:(2,1), B:(1,1), C:(1,2)$; a trail is $A–B–C–A–C$. Undirected degree alone does not decide the directed problem.

Zero-edge trails are allowed. If E is empty and V nonempty, any single-vertex trail covers all edges. If V is empty there is no start, and no Euler circuit under this convention. At $m > 0$, extra isolated vertices do not affect traversal of edges.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Seven-edge multigraph and one repair

On A, B, C, D take two separate AD IDs, two BD IDs, and CD, AC, BC once each. The graph is connected with degrees 3,3,3,5, so four odd vertices prevent an Euler trail.

Add t=AB: degrees become 4,4,3,5, leaving C, D odd. The trail C–A–D–A–B–D–B–C–D uses eight edges. Its two AD traversals use different IDs, as do its two BD traversals; AB, AC, BC, CD each occur once. Endpoints are precisely C, D. Using endpoint pairs as used keys would incorrectly suppress the second parallel AD or BD; edge IDs preserve distinct connections.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

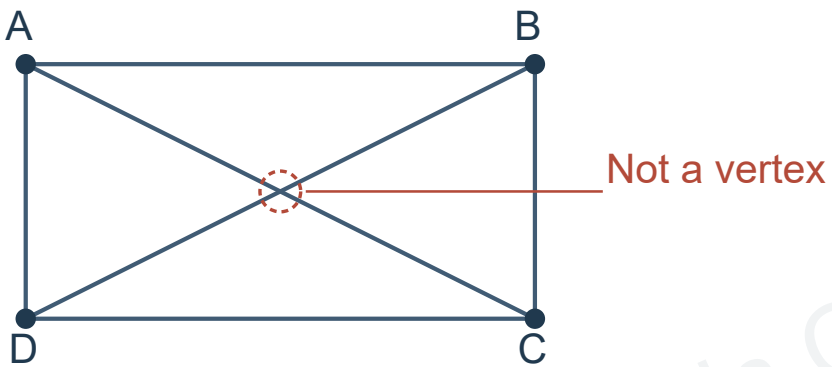
Planarity versus a plane embedding

An abstract graph records only vertices and adjacency; a drawing places them in the plane. A graph is planar if some drawing avoids crossings away from endpoints. Such a drawing is a plane embedding.

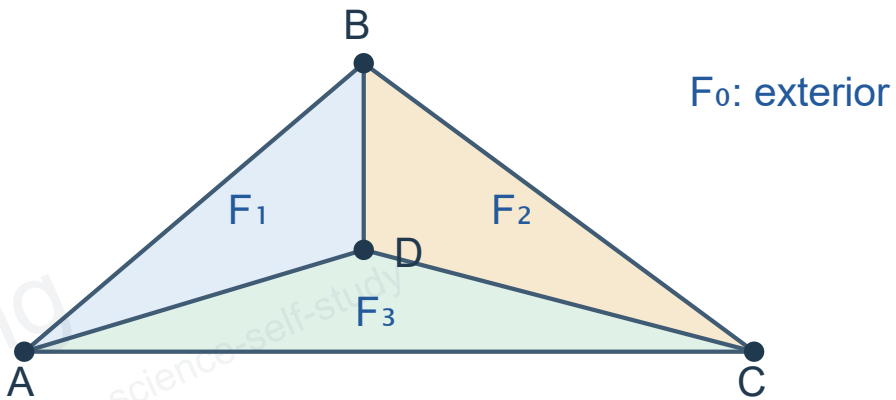
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Planarity versus a plane embedding (continued)

The same K_4 redrawn without crossings



V={A,B,C,D} ; E={AB,AC,AD,BC,BD,CD}



Right: 4 faces; 4−6+4=2

Both drawings have four vertices, six edges, and degree three everywhere. The left crossing is not a fifth vertex. Placing D inside outer triangle ABC gives inner faces F_1,F_2,F_3 and outer F_0. A crossing in one drawing does not prove nonplanarity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler's formula by cycle-edge deletion

A connected plane graph without cycles is a tree: $e=v-1, f=1$, so $v-e+f=2$. Otherwise delete a cycle edge. Connectivity remains, its two distinct incident faces merge, and both e, f decrease by one. Continue to a tree; the invariant proves the formula. Deleting an arbitrary bridge would disconnect the graph, and its sides belong to one face rather than two.

Operation	v	e	f including exterior	v−e+f
K_4	4	6	4	2
Delete AD; merge F_1, F_3	4	5	3	2
Delete BD; merge inner faces	4	4	2	2
Delete AC; merge inner/outer	4	3	1	2

Remaining AB, BC, CD form a tree. With c components, add $c-1$ noncrossing bridges without changing faces, then apply the connected formula to obtain $v-e+f=1+c$. Components share one outer face.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Face-boundary multiplicities and edge bounds

Each embedded edge has two sides. A bridge is counted twice in the same face boundary, so $\Sigma \ell(F)=2e$ even with bridges; face boundaries need not be simple cycles.

In a connected simple plane graph with $v \geq 3$, every face boundary has length at least three. Length one requires a loop; length two requires parallel edges or an entire one-edge graph. If there are cycles, boundary cycle portions exclude these cases. If the graph is a tree, its sole boundary has length $2(v-1) \geq 4$. Thus $3f \leq 2e$ and

$$3(2 - v + e) \leq 2e \quad \Rightarrow \quad e \leq 3v - 6.$$

K_5 has $v=5, e=10>9$ and is nonplanar. Satisfying the inequality is only necessary, not sufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Girth and bipartite nonplanarity

Girth g is shortest cycle length. A connected simple plane graph containing a cycle of girth $g \geq 3$ has each face boundary length at least g. Bridge excursions lengthen boundaries without shortening a cycle portion. Hence $gf \leq 2e$ and

$$e \leq \frac{g}{g-2}(v-2).$$

A forest has no finite girth; use $e=v-c$ rather than substituting infinity into this proof. A simple bipartite planar graph with $v \geq 3$ has $g \geq 4$ when cyclic, hence $e \leq 2v-4$. A tree satisfies $v-1 \leq 2v-4$ too. $K_{3,3}$ has $v=6, e=9>8$, proving nonplanarity despite passing the weaker bound. A proof must exclude all embeddings, not one drawing.

For a disconnected triangle-free simple planar graph with $v \geq 3$, place components in disjoint disks and join them in a chain with $c-1$ noncrossing bridges. No new cycle or triangle arises. The connected bound gives $e+c-1 \leq 2v-4$, hence $e \leq 2v-4$ for the original. The general girth proof still retains its cycle premise.

Plane duals depend on the embedding

For each face F of a connected plane graph create vertex F*. Each original edge e gives a dual edge across e between its two incident faces. If these are the same face, the dual edge is a loop, as for an original bridge. Preserve parallel dual edges. Thus v*=f, e*=e, f*=v; both graphs satisfy Euler's formula. This changes edge incidence, not merely matrix row/column order.

For embedded K_4, F_0=ABC, F_1=ABD, F_2=BCD, F_3=CAD:

Edge	AB	BC	CA	AD	BD	CD
Dual endpoints	0–1	0–2	0–3	1–3	1–2	2–3

The dual is again K_4. Adding a leaf X at D inside an existing face makes DX a bridge; the dual gains a loop at that face, not two new faces.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why does a face boundary traverse a bridge twice?

Fix triangle ABC, put leaf D inside it, and add bridge CD. Number the edges $e_1 = AB$, $e_2 = BC$, $e_3 = CA$, $e_4 = CD$. There are just the exterior F_0 and interior F_1, so $(v, e, f) = (4, 4, 2)$. The bridge does not split the interior: a curve can pass around its leaf endpoint D to the other side.

Face	One complete boundary walk	Edge IDs	Length
F_0	$A \to C \to B \to A$	3, 2, 1	3
F_1	$A \to B \to C \to D \to C \to A$	1, 2, 4, 4, 3	5

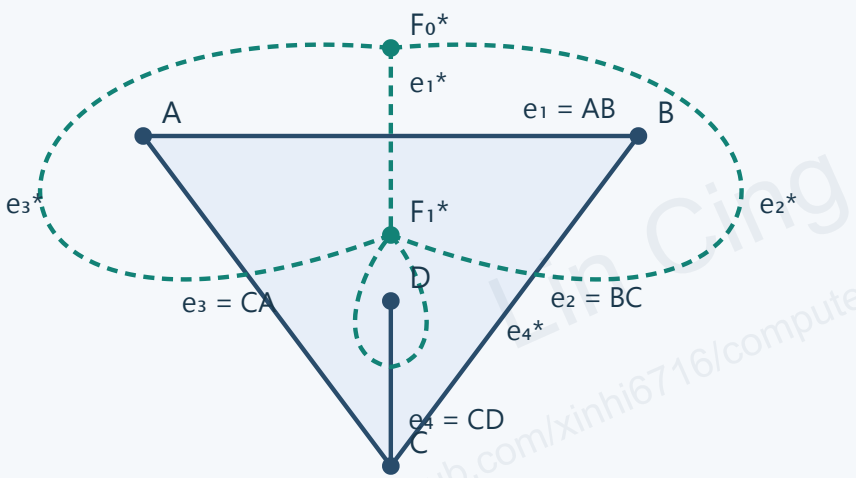
Boundary length counts occurrences. The interior walk uses the single bridge to reach D and returns on its other side. Thus $3 + 5 = 8 = 2e$. This boundary repeats C and CD and is not a simple cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Draw each primal edge with its matching dual edge

Choose dual vertices F_0^*, F_1^* inside the two faces. Each dashed e_i^* crosses solid e_i once. A crossing is not a new vertex.

Primal and dual: each dashed edge crosses its matching edge once



Solid blue: primal G

Dashed teal: dual G*

F_0: exterior of triangle

F_1: interior of triangle

D lies in F_1; CD is a bridge.

e_4^* surrounds D; both ends are F_1^*.

Crossings are not vertices. Dual vertices are chosen points inside faces.

Each triangle edge separates the same two faces, so their dual edges share endpoints. Both sides of CD belong to the interior. Its dual starts and ends at F_1^*, giving a loop.

Read the dual edge list from face incidences

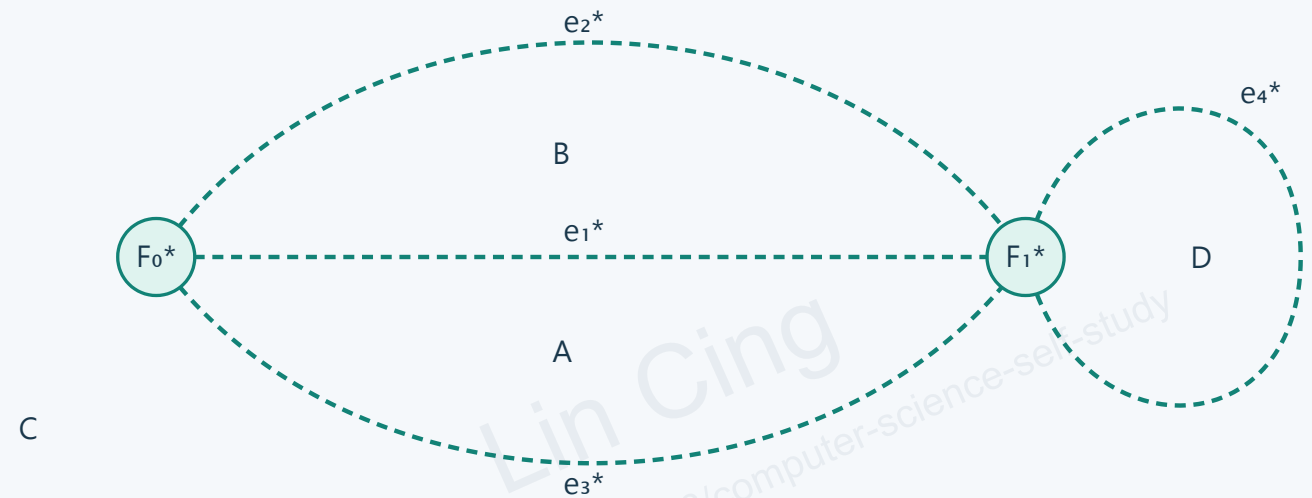
Primal edge	Incident faces	Dual edge	Contribution to $\deg(F_0^*)$	Contribution to $\deg(F_1^*)$
$e_1 = AB$	F_0, F_1	$F_0^* F_1^*$	1	1
$e_2 = BC$	F_0, F_1	Same endpoints, distinct ID	1	1
$e_3 = CA$	F_0, F_1	Same endpoints, distinct ID	1	1
$e_4 = CD$	F_1, F_1	Loop at F_1^*	0	2

The degrees are $3, 5$, matching the two boundary lengths. The degree sum is $3 + 5 = 2 \cdot 4$. **A loop contributes two edge ends to one vertex; parallel edges remain distinct.** Collapsing the three parallel edges or deleting the loop destroys the edge-by-edge dual construction.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A simple primal graph can have a nonsimple dual

The dual alone: three parallel edges and one loop



A, B, C, D label four dual faces. A loop contributes two to degree.

Primal vertex degrees are $2, 2, 3, 1$; the dual has two vertices of degrees $3, 5$. Labels A, B, C, D mark the four dual faces, not additional vertices. Its Euler count is $v^* - e^* + f^* = 2 - 4 + 4 = 2$. Simplicity of the primal does not exclude dual loops or parallel edges.

Deleting a cycle edge differs from deleting a bridge

Each row independently starts from the complete triangle with its interior leaf. Keep all four vertices.

Operation	Components c	Faces f	$v - e + f$	Reason
No deletion	1	2	$4 - 4 + 2 = 2$	Triangle separates inside from outside
Delete AB	1	1	$4 - 3 + 1 = 2$	Remaining graph is a tree; the faces merge
Delete CD	2	2	$4 - 3 + 2 = 3$	D is isolated; the triangle still separates faces

The last row needs $v - e + f = 1 + c$. **Removing CD cannot merge distinct faces:** its two sides already belonged to one face. An isolated point encloses no new region; a new component does not bring another exterior face.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why does a connected primal graph have $f^* = v$?

Fix a connected plane embedding, allowing bridges. One dual vertex per face and one dual edge per primal edge give $v^* = f$ and $e^* = e$.

The dual is connected. A curve joining points in any two faces can avoid the finite vertex set and cross edges transversely finitely many times. Reading successive crossed faces yields a walk in the dual. Euler's formula therefore applies to it:

$$f^* = 2 - v^* + e^* = 2 - f + e = v.$$

The last equality uses primal Euler count $v - e + f = 2$. The bridge-loop correspondence also works in reverse: a nonbridge lies on a simple cycle. The Jordan-curve property separates that cycle's two sides, so the edge cannot have the same face on both sides. Thus a primal edge is a bridge exactly when its dual edge is a loop.

For face-boundary multiplicity and Euler's formula, see Grimaldi, fifth edition, Section 11.4, printed pp. 545–547. The same embedding connects every count in this worked example.

Nonplanarity criteria

Kuratowski's theorem states that a finite graph is planar iff it contains no subdivision of K_5 or $K_{3,3}$. Subdivision replaces edges by internally vertex-disjoint paths, inserting degree-two vertices. Wagner's theorem uses minors instead, allowing vertex/edge deletion and edge contraction.

The sufficiency of these characterizations is cited here, not derived from Euler bounds. Necessity follows because deletion preserves a plane drawing, contraction inside a narrow edge neighborhood adds no crossing, and suppressing a degree-two intermediate vertex also preserves planarity. If these operations produce a known nonplanar obstruction, the original was nonplanar. One found obstruction suffices; unsuccessful manual searching does not prove its absence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Necessary Hamilton-cycle conditions

A Hamilton path visits each vertex once; a Hamilton cycle also closes, with $n \geq 3$ in a simple graph. Every vertex then has degree at least two, and no bridge or cut vertex is possible. These conditions alone are insufficient.

More generally, for every nonempty vertex set S,

$$c(G - S) \leq |S|.$$

Deleting |S| vertices from the cycle leaves at most |S| path segments; extra edges in G−S can join them but not increase component count. For S=V the count is zero. In $K_{2,3}$, deleting the two-vertex part leaves three isolates, violating $3 \leq 2$, so no Hamilton cycle exists. Yet $b_1–a_1–b_2–a_2–b_3$ is a Hamilton path. Euler questions require their separate degree checks.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Ore's edge-addition lemma

For n≥3, nonadjacent u, v with d(u)+d(v)≥n, G has a Hamilton cycle iff G+uv does. The forward direction is immediate. If a cycle in G+uv avoids uv it already belongs to G. Otherwise deleting uv leaves Hamilton path v_1=u, v_2,…,v_n=v. On indices 1,…,n−1 define

$$A = \{i : uv_{i+1} \in E\}, \qquad B = \{i : v_i v \in E\}.$$

Because uv is absent, |A|=d(u),|B|=d(v). Disjointness would bound their sum by n−1, a contradiction. At a common i, the original edges give cycle

$$v_1, v_2, \ldots, v_i, v_n, v_{n-1}, \ldots, v_{i+1}, v_1.$$

This uses two reconnections along a vertex order, without a planarity assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ore and Dirac sufficient conditions

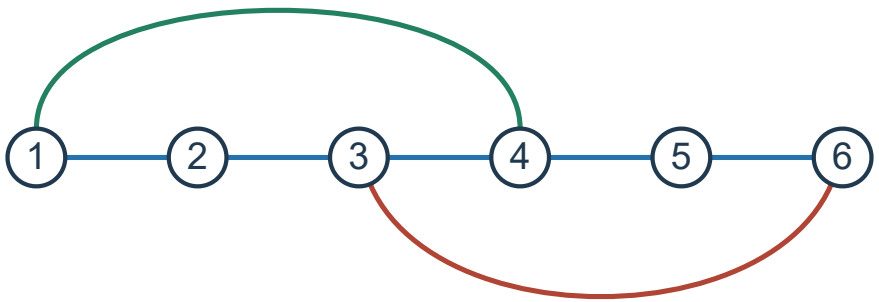
Ore's theorem assumes $d(u)+d(v) \geq n$ for every nonadjacent pair. Add missing edges one by one. Degrees only increase, preserving the premise for remaining pairs; the final K_n has a Hamilton cycle. Applying the lemma backward proves G does too. Dirac's condition $\delta(G) \geq n/2$ implies Ore immediately.

Both are sufficient. Failure does not prove nonexistence: C_6 has degree $2<3$ yet is itself a Hamilton cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ore and Dirac sufficient conditions (continued)

Ore's lemma: reverse the path suffix to avoid the new edge



Original path: 1 — 2 — 3 — 4 — 5 — 6

At i=3,existing reconnections are

1—4 (green),3—6 (red)

Cycle: 1 → 2 → 3 → 6 → 5 → 4 → 1

Follow 1→2→3,then 6; reverse through 6→5→4 and return to 1.

At i=3, take 1–2–3, edge 3–6, reversed segment 6–5–4, and 4–1. Each vertex occurs once without using the added endpoint edge 1–6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hamilton paths from a cycle theorem

For $n \geq 2$, $d(u)+d(v) \geq n-1$ for all nonadjacent pairs ensures a Hamilton path. Add a universal new vertex z, obtaining $n+1$ vertices. Nonadjacent pairs remain original pairs and gain degree sum two, reaching at least $n+1$. Ore gives a cycle in the new graph. Delete z and its two incident cycle edges to leave a Hamilton path in G. In particular $\delta(G) \geq (n-1)/2$ suffices.

At $n=2$, an edgeless pair fails the premise $0 \geq 1$; an edge itself is the required path. This gives existence, not shortestness or prescribed endpoints. Fixed endpoints require another check; deleting z does not freely choose them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every tournament has a directed Hamilton path

A tournament has exactly one directed arc between every two distinct vertices. It has a Hamilton path but need not have a cycle; the transitive orientation $i \to j$ iff $i < j$ is acyclic.

Induct on vertices. Given path $v_1 \to \ldots \to v_{n-1}$, add x. If $x \to v_1$, prepend it. Otherwise $v_1 \to x$. If a first j with $x \to v_j$ exists, minimality gives $j \geq 2$ and $v_{j-1} \to x$, so insert x there. If none exists, every $v_i \to x$ and append it. Each case preserves a directed path containing all vertices.

For $a \to b \to c \to d$ with $a \to x$, $b \to x$, $x \to c$, $d \to x$, insert between b, c to get $a \to b \to x \to c \to d$. x need not dominate every later vertex.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex coloring

A proper k-coloring maps vertices to $\{1,\ldots,k\}$ with different colors at adjacent vertices. The chromatic number $\chi(G)$ is minimum k. A nonempty edgeless graph has $\chi=1$; empty vertices have $\chi=0$ by convention. A loop prevents proper coloring.

The bounds are $\omega(G)\leq\chi(G)\leq\Delta(G)+1$, not universal equalities. Odd cycles need three colors; bipartite nonempty graphs need at most two. Greedy coloring depends on vertex order and need not minimize χ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bipartiteness, odd cycles, and greedy coloring

Two-colorability is equivalent to having no odd cycle. Alternating colors along a cycle forces even length. Conversely BFS each component and color by distance parity. A same-color edge uv together with tree paths from their deepest common ancestor forms an odd cycle, contradiction. Nonempty edgeless graphs need only one color.

Greedy assigns the smallest positive color absent from already colored neighbors. At most Δ colors are forbidden, proving the Δ+1 upper bound, not optimality. On path 1–2–3–4, order 1,4,2,3 gives colors 1,1,2,3; order 1,2,3,4 gives 1,2,1,2. Three greedy colors do not prove χ=3. An edge gives lower bound two and the exhibited coloring upper bound two, so χ=2. The four-color theorem for simple planar graphs is cited, not proved by a face-count inequality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chromatic polynomials

P_G(k) counts proper colorings with k named colors. For a nonloop edge e,

$$P_G(k) = P_{G-e}(k) - P_{G/e}(k).$$

Contraction identifies endpoints and removes e. Parallel edges may merge without changing constraints; an unrelated remaining loop makes the polynomial zero and cannot be silently deleted. Contracting a simple graph this way leaves no loop.

An n-vertex tree has $k(k-1)^{n-1}$ colorings; K_n has $k(k-1)\dots(k-n+1)$. Values at nonpositive k can be evaluated algebraically but are not direct coloring counts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deletion–contraction by a bijection

Partition proper colorings of G−e, e=uv, by whether u, v agree. Different colors give exactly G-colorings; equal colors biject with G/e-colorings. Thus $P_{G-e} = P_G + P_{G/e}$. Inducting on edges from edgeless polynomial k^n proves polynomiality. A simple graph's first terms are $k^n - ek^{n-1}$: each deletion retains leading one while its contraction subtracts leading k^{n-1} once per edge.

For C_4, deletion gives a four-vertex tree and contraction a triangle:

$$P_{C_4}(k) = k(k-1)^3 - k(k-1)(k-2) = k^4 - 4k^3 + 6k^2 - 3k.$$

Values at two and three are 2,18. Named-color swaps count as different colorings, unlike Chapter 16's symmetry orbits.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Labeled graphs and components

On n fixed labels every unordered pair is an optional edge, giving $2^{\binom{n}{2}}$ graphs and $\binom{\binom{n}{2}}{m}$ with m edges. Do not divide uniformly by n!: automorphism counts differ.

Let c_n count connected labeled graphs, $c_1=1$. Classify uniquely by the size k of vertex one's component. Choose its k−1 other labels, a connected graph there, and any graph on the rest; cross edges are absent:

$$2^{\binom{n}{2}} = \sum_{k=1}^{n} \binom{n-1}{k-1} c_k 2^{\binom{n-k}{2}}.$$

Thus $c_2=2-1=1$,$c_3=8-(2+2)=4$,$c_4=64-(8+6+12)=38$. The subtracted terms at four correspond to k=1,2,3, leaving the connected k=4 case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inclusion–exclusion for no isolated vertices

If specified j vertices are isolated, every incident edge is forbidden; only edges among the remaining n−j vertices are free. Hence

$$N_n = \sum_{j=0}^{n} (-1)^j \binom{n}{j} 2^{\binom{n-j}{2}}.$$

At n−j=0 or 1 the binomial exponent is zero and one edgeless graph remains. At n=4, 64−4 • 8+6 • 2−4 • 1+1=41. This exceeds 38 connected graphs by three perfect matchings, each two disjoint edges. No isolates is substantially weaker than connectivity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries and further exercises

Euler and Hamilton properties imply neither one another. Count the outer face. Isomorphism needs adjacency correspondence, not only degrees. Three-colorability does not require a triangle. Chromatic-polynomial coefficients may be negative; color choices are not independent. The exercise and solution PDFs give full derivations. Chapter 12 builds DFS trees and cut vertices; Chapter 13 adds weights, shortest paths, and flows to the same model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study